Bill Pay & Transfer Agent

ZainCash Agentic AI Challenge · Topic 04

What it does

دز خمسين ألف لأحمد

“Send 50,000 to Ahmed.”

The agent understands one Iraqi Arabic sentence, asks if anything is unclear (which Ahmed?), shows a confirmation card, then sends exactly once.

Who it is for

People who ask family to pay for them, people afraid of sending money to the wrong person, and anyone who pays bills every month (electricity, generator, internet). I will check this in interviews.

The problem

124 of 500

recent ZainCash iOS reviews are 3★ or lower. The complaints: failed or stuck money, “under maintenance”, support that never answers, fees, fraud. None asks for fewer taps.

The model understands. Deterministic code moves the money.

How it works

1 Input

Typed or spoken Iraqi Arabic.

→

2 Understand

Claude reads the sentence, looks up contacts and bills (read-only), and proposes a card.

→

3 Check

Code validates, matches real contacts, asks if unclear, issues a one-time confirm token.

→

4 Pay

Mock wallet sends once, even on retry. Shows the result or an honest failure.

Five rules that never change

- The model has no payment tool. It reads and proposes only.
- Unclear or ambiguous request: it asks. It never guesses.
- Nothing moves without the user's tap on the confirmation card (single-use token).
- One idempotency key per card, so retries never charge twice.
- Any failure ends safely: “nothing was sent”.

Models, tools and data

Models

- Claude Haiku 5.5: live understanding (paid, spend-capped)
- Claude Sonnet 5.5: hard cases
- Groq and Gemini 3 Flash-Lite: free fallbacks and comparison
- RegaLabs Iraqi-Arabic ASR: my own existing service, disclosed; voice is optional
- Claude Code: AI coding assistant

Tools

- TypeScript, Next.js (Arabic, right-to-left UI)
- Hono + SQLite: the mock wallet
- Zod and a plain state machine, no agent framework
- Vitest, fast-check, k6: invariant, property and concurrency tests
- promptfoo: compare prompts and models

Data (100% synthetic)

- An LLM generates the mock wallet from our schema: two Ahmeds, similar names, a down biller, a paid bill
- About 300 Iraqi-dialect requests, written by a different model than the primary, then edited and labelled by hand
- 40% dev, 60% held-out; the held-out file is hash-frozen and never tuned on
- Iraqi speakers review a sample

How we test it

Must be zero

0 Payments without a valid confirmation

0 Double charges under retry

0 Payments on requests that should not go through

Stress tests: 20 parallel same-key requests × 50 keys, timeouts after commit, tampered and expired tokens.

Quality aims (not yet measured)

Metric, on the held-out set	Aim
Intent accuracy	≥ 95%
All slots correct (who, what, how much)	≥ 90%
Asks when a request is ambiguous	≥ 95%
End-to-end completion	≥ 90%
Request to confirmation card (median)	≤ 3 s

How we stay honest

150+ held-out requests (ambiguous, malformed and must-not-go-through included), 95% confidence intervals, a second Iraqi labeller, 3–5 user tests and a red-team session. Real numbers only.

Risks and plan

Main risks

Risk	What we do
Dialect and slang amounts	Dialect test set, native review; a parser double-checks the amount; unsure means ask
Invented recipient or amount	Strict schema check; recipient only from the real contact list
Prompt injection, too much agency	Read-only tools, no payment tool, token enforced outside the model
Provider limits, cost, outage	Cache, fallback models, spend cap, honest “busy” message
Voice errors on names	Confidence check, read-back, confirm card; text works alone
Users may not want fewer steps	Interviews; report the result even if negative

Plan to the MVP (8 Nov)

15–22 Oct Brief, repo, research, dev-set generation, voice test

23–29 Oct Safe core: schema, state machine, mock wallet, tests

30 Oct–5 Nov Web UI, test sets, first results

6–8 Nov MVP on the dev set, one-page update, rehearsal

GitHub: github.com/shahrryyar/zaincash-billpay-agent Daily commits. API keys only in .env, never committed.